Deep Modules, per Matt Pocock

What they are, why they matter for TypeScript and AI-assisted coding, and how to find and deepen shallow ones

Compiled from the Matt Pocock brain

2026-10-24

Contents

About this report

1. What a deep module is
   1.1 The definition
   1.2 Origins: Ousterhout, and where Matt departs from him
   1.3 Four principles
2. Why it matters
   2.1 For AI-assisted coding
   2.2 For TypeScript codebases
3. Finding shallow modules
   3.1 The friction questions
   3.2 Other signals in his skills
4. Deepening
   4.1 The process Matt's skill follows
   4.2 The human's role and cadence
   4.3 Gray boxes and their limits
5. Seams and testability
6. Interface design
7. Worked TypeScript examples
   7.1 Collapsing a pass-through cluster
   7.2 A seam that earns its keep
   7.3 Narrowing the state a caller must handle
   7.4 A reducer as a deep module for UI state
8. Anti-patterns
9. Checklist
10. Dates
11. Limits of this report

About this report

This report sets out Matt Pocock's documented position on deep modules, drawn from the Matt Pocock brain (`brains/matt-pocock`). It is written about him, in the third person; it is not written as him and does not claim his endorsement.

Every passage is one of three kinds, and the kinds are kept apart:

- **What Matt says**, with a bracketed citation in the brain's form: a slug and heading (joined by §) for a skill or written note, or a slug and timestamp (joined by @) for a reviewed transcript. The slug names a note in the brain's `corpus/`.
- **Synthesis**, which joins several cited passages and is marked *Synthesis:*.
- **Inference** for a TypeScript codebase, marked *Inference:*. Inferences are this report's application of his views, not things he said. All TypeScript examples are inference unless a citation says otherwise.

Two cautions apply throughout. Nearly all of the material dates from February to October 2026, in videos and skill files about AI-assisted coding, and the corpus has no long-form article or book by Matt on module design. And the skill files are an October 2026 snapshot that says what each skill instructs, not necessarily his current view [ask-matt § The main flow: idea → ship].

1. What a deep module is

1.1 The definition

Matt's `codebase-design` skill states the goal in one line: deep modules, meaning a lot of behaviour behind a small interface, placed at a clean seam and testable through that interface, for the sake of leverage for callers, locality for maintainers and testability for everyone [codebase-design § Glossary].

The skill defines its terms precisely and tells the agent to use them exactly, without substituting "component", "service", "API" or "boundary" [codebase-design § Glossary]:

Term	Meaning in the skill
Module	Anything with an interface and an implementation. Scale-agnostic: a function, class, package or tier-spanning slice.
Interface	Everything a caller must know to use the module correctly: the type signature, but also invariants, ordering constraints, error modes, required configuration and performance characteristics.
Implementation	What is inside the module. Distinct from an adapter: a Postgres repository is a small adapter with a large implementation; an in-memory fake is the reverse.

Term	Meaning in the skill
Depth	Leverage at the interface: how much behaviour a caller or test can exercise per unit of interface they must learn.
Seam	Credited to Michael Feathers: a place where behaviour can be altered without editing in that place; the location where the interface lives.
Adapter	A concrete thing that satisfies an interface at a seam. It names a role, not a substance.
Leverage	What callers get from depth: more capability per unit of interface, paid back across many call sites and tests.
Locality	What maintainers get from depth: change, bugs, knowledge and verification concentrate in one place.

A module is **deep** when a large amount of behaviour sits behind a small interface and **shallow** when the interface is nearly as complex as the implementation [codebase-design § Glossary]. The skill's picture is a small interface (few methods, simple parameters) over a deep implementation that hides complex logic, set against a large interface over a thin implementation that just passes calls through [codebase-design § Deep vs shallow].

In the de-slop video Matt adds that the interface also includes nebulous usage information such as documentation [how-to-de-slop-a-codebase-ruined-by-ai @ 00:02:22], and gives module examples ranging from a set of React components forming a page, to the authentication functions, to a logger [how-to-de-slop-a-codebase-ruined-by-ai § Detailed source notes]. His example of a high-quality deep module is a well-regarded open-source library such as TanStack Query, which hides a lot behind a simple interface [how-to-de-slop-a-codebase-ruined-by-ai § Detailed source notes].

1.2 Origins: Ousterhout, and where Matt departs from him

Matt credits John Ousterhout, author of *A Philosophy of Software Design*, for the deep-versus-shallow idea itself [how-to-de-slop-a-codebase-ruined-by-ai § Detailed source notes]. He borrows two more Ousterhout framings: **complexity**, any structure that makes a system hard to understand and modify [software-fundamentals-matter-more-than-ever § Detailed source notes], and **tactical versus strategic programming**, where tactical work gets the current task working and strategic work invests in design so the next change is easier [your-agent-is-a-tactical-tornado-vo33ex § Detailed source notes]. He recommends the book, along with the first chapters of *Domain-Driven Design* and *The Pragmatic Programmer* [ai-skills-with-matt-pocock § Detailed source notes].

His skill deliberately departs from the book in one place. It rejects depth as a ratio of implementation lines to interface lines, which it labels Ousterhout's framing, because that rewards padding the implementation; it uses depth-as-leverage instead [codebase-design § Rejected framings]. It also rejects reading "interface" as the TypeScript `interface` keyword or a class's public methods, and rejects "boundary" because DDD already uses it for bounded context [codebase-design § Rejected framings].

The other borrowed ideas are credited too: the seam to Michael Feathers [codebase-design § Glossary], software entropy to *The Pragmatic Programmer* [software-fundamentals-matter-more-than-ever § Detailed source notes], and investing in design every day to Kent Beck [software-fundamentals-matter-more-than-ever § Detailed source notes]. He presents the package as old advice reapplied, not a new AI method [your-codebase-is-not-ready-for-ai § Detailed source notes].

Synthesis: the vocabulary is Matt's packaging of other authors' ideas, so it should not be read as Ousterhout's, Feathers's or Beck's own views.

1.3 Four principles

The skill adds four rules [codebase-design § Principles]:

1. **Depth belongs to the interface, not the implementation.** A deep module may be built from small, mockable, swappable parts as long as they are not part of the interface. It can have internal seams for its own tests as well as the external seam at its interface.
2. **The deletion test.** Imagine deleting the module. If complexity vanishes, it was a pass-through. If complexity reappears across many callers, it was earning its keep.
3. **The interface is the test surface.** Callers and tests cross the same seam, so wanting to test past the interface suggests the module is the wrong shape.
4. **One adapter means a hypothetical seam; two adapters means a real one.** Do not introduce a seam unless something actually varies across it.

2. Why it matters

2.1 For AI-assisted coding

Matt's central claim is that software fundamentals matter more with AI, not less, because bad code is the most expensive it has ever been when an agent has to work in a codebase that is hard to change [software-fundamentals-matter-more-than-ever § Detailed source notes]. He reached it after a "specs to code" experiment in which the code got progressively worse with each pass [software-fundamentals-matter-more-than-ever § Detailed source notes], and he calls ignoring the code "vibe coding by another name" [software-fundamentals-matter-more-than-ever @ 00:02:06].

He gives four reasons deep modules help agents in particular.

- **Navigation.** A codebase of many tiny shallow modules is hard for the AI to explore; it may not reach the right module or understand the dependencies [software-fundamentals-matter-more-than-ever @ 00:13:26]. He wants the file system to match the developer's mental map, because an agent starts with no memory and sees only a jumble of exports, and he wants public types visible so the agent reads the interface first and opens the implementation only if needed, which he calls progressive disclosure [your-codebase-is-not-ready-for-ai § Detailed source notes].
- **Memory.** A human remembers how to work around a bad codebase; an agent must be re-taught every session, like the amnesiac in *Memento* [do-software-fundamentals-still-matter § Detailed source notes]. He calls it "twenty new starters a day" [your-codebase-is-not-ready-for-ai @ 00:06:35].

- **Testing and feedback.** Good codebases are easy to test, so a better codebase gives the agent better feedback [software-fundamentals-matter-more-than-ever @ 00:12:12]. Deep modules mean fewer structure-sensitive tests, because tests target only the interface [fixing-the-pr-bottleneck-matt-pocock-aihero § Detailed source notes].
- **Division of labour.** The human designs and verifies the interface; the agent handles what is behind it [your-codebase-is-not-ready-for-ai § Detailed source notes]. The slogan is "design the interface, delegate the implementation" [software-fundamentals-matter-more-than-ever § Detailed source notes].

He ranks the codebase above prompts and the AGENTS.md file as an influence on output [your-codebase-is-not-ready-for-ai @ 00:00:03], and says a clean codebase lets cheaper models and fewer tokens do the work [matt-pocock-s-agentic-engineering-workflow-just-copy-him § Detailed source notes]. He also treats "deep modules" as a *leading word*: the term is in the model's training, so naming it in a prompt makes the model use it back [ai-skills-with-matt-pocock @ 00:55:19].

Why agents do not produce deep modules on their own: he says they behave like Ousterhout's "tactical tornado", a fast developer who leaves a mess behind [your-agent-is-a-tactical-tornado-vo33ex § Detailed source notes], partly because they are rewarded on tactical benchmarks, which he flags as his reading of incentives [tell-your-agents-to-care-about-the-codebase § Detailed source notes]. His remedy is to be the "annoying colleague" who asks for the refactor first, and to tell the agent the codebase should improve over time [tell-your-agents-to-care-about-the-codebase § Detailed source notes].

These are his observations, not measured outcomes.

2.2 For TypeScript codebases

Matt's published work in this corpus doesn't connect deep modules to TypeScript language features in a single argument. It does contain adjacent TypeScript positions that fit the picture:

- An object with `status`, `error` and `data` properties allows states that should not exist; the fix is a discriminated union of object types [you-have-to-know-this-typescript-pattern @ 00:00:02] [you-have-to-know-this-typescript-pattern @ 00:00:49].
- In 2023-12 he said shared code inside an application, such as a utils folder, can be its most complex layer to type [application-vs-library-ts-typescript § Detailed source notes].
- In October 2026 he praised reducers for front-end work with agents: a reducer takes an action and returns the next state, can be tested outside the browser, and keeps the React code simple [the-secret-to-building-robust-frontend-apps-with-agents § Detailed source notes].

Inference: in TypeScript, the type signature is the part of the interface the compiler can enforce, so a deep module's exported types are where an agent should start reading. Narrow exported types (unions over bags of optional fields, a few named functions over a class with many public methods) shrink what a caller must learn. The skill's "interface" is still wider than the `interface` keyword: invariants and error modes count even when no type expresses them [codebase-design § Rejected framings].

Inference: a `utils/` folder of tiny re-exported helpers is the TypeScript shape most likely to fail the deletion test, because each helper's interface is about as large as its body.

3. Finding shallow modules

3.1 The friction questions

Matt's `improve-codebase-architecture` skill does the survey. It starts with a YAGNI scoping rule: deepening pays off by easing future change, so weight recently changed code; take the user's direction if one was given, otherwise read commit history for hot spots. It reads `GLOSSARY.md` and ADRs first, then has a sub-agent walk the code noting friction rather than following rigid heuristics. The friction questions are [improve-codebase-architecture § Process › 1. Explore]:

- Where does understanding one concept require bouncing between many small modules?
- Where is a module shallow, with an interface nearly as complex as its implementation?
- Where have pure functions been extracted just for testability, while the real bugs hide in how they are called (a failure of locality)?
- Where do tightly coupled modules leak across their seams?
- Which parts are untested, or hard to test through their current interface?

Anything suspected of being shallow gets the deletion test: would deleting it concentrate complexity, or just move it [improve-codebase-architecture § Process › 1. Explore]?

3.2 Other signals in his skills

- **No correct seam during bug fixing.** If no seam lets a regression test exercise the real bug pattern at the call site, the `diagnosing-bugs` skill says that is itself the finding: the architecture is preventing the bug from being locked down [diagnosing-bugs § Phase 5: Fix + regression test].
- **Tests that break on refactor.** The `tdd` skill's tell for an implementation-coupled test is one that breaks when behaviour has not changed [tdd § Anti-patterns].
- **Review smells.** The `code-review` skill carries Shotgun Surgery (gather what changes together), Divergent Change (split a module edited for unrelated reasons), Middle Man and Primitive Obsession [code-review § Process › 3. Identify the standards sources].
- **Rules duplicated across tiers.** In the de-slop video the candidate he picks is an insertion rule implemented separately on front end and back end, which could drift; the fix gains locality by putting the rule in one module [how-to-de-slop-a-codebase-ruined-by-ai § Detailed source notes].

Synthesis: these signals point at one failure: behaviour that belongs together is spread across callers, so knowledge and verification are not local.

4. Deepening

4.1 The process Matt's skill follows

1. **Explore** recent hot spots with the friction questions (section 3.1).
2. **Report** candidates as a self-contained HTML file in the OS temp directory, so nothing lands in the repo. Each candidate has files, problem, solution, benefits in locality, leverage and testing, a before/after diagram, and a strength of Strong, Worth exploring or Speculative; the report

ends with a top recommendation. It does not propose interfaces yet; it asks which candidate to explore [improve-codebase-architecture § Process › 2. Present candidates as an HTML report].
3. **Grill** the chosen candidate over constraints, dependencies, the shape of the deepened module, what sits behind the seam, and which tests survive [improve-codebase-architecture § Process › 3. Grilling loop]. Alternative interfaces can be explored with the design-it-twice pattern of parallel sub-agents [codebase-design § Going deeper].
4. **Record** as you go: a new name goes into `GLOSSARY.md`, and a load-bearing rejection is offered as an ADR so later reviews do not re-suggest it [improve-codebase-architecture § Process › 3. Grilling loop].

A candidate that contradicts an existing ADR is surfaced only when the friction justifies revisiting it, and is marked as such [improve-codebase-architecture § Process › 2. Present candidates as an HTML report].

4.2 The human's role and cadence

It is not an away-from-keyboard skill: the agent is the sergeant who finds candidates and the human the general who decides what is good for long-term health. He suggests running it every couple of days on a fast-moving codebase. On a legacy codebase, which he says in practice means one with many shallow modules, the first need is a harness of tests around deep modules before changes are made [how-to-de-slop-a-codebase-ruined-by-ai § Detailed source notes]. Module boundaries, and how their interfaces change, belong in the PRD [software-fundamentals-matter-more-than-ever § Detailed source notes].

4.3 Gray boxes and their limits

A deepened module becomes a gray box: the implementation can still be opened for performance or taste, but routine changes rely on the interface and the tests [your-codebase-is-not-ready-for-ai § Detailed source notes]. He attaches conditions: it is for less critical modules (he gives finance as a case where he would still review the implementation), and it needs a testable boundary outside the module [software-fundamentals-matter-more-than-ever @ 00:15:53]. It is "still a million miles away" from vibe coding, because the human decides what goes in each module and designs the boundaries [your-codebase-is-not-ready-for-ai @ 00:05:50].

5. Seams and testability

The skill's three rules for testable interfaces: accept dependencies instead of creating them, return results instead of producing side effects, and keep the surface small, since fewer methods means fewer tests and fewer parameters means simpler setup. Its examples are TypeScript: pass a payment gateway into `processOrder` rather than constructing a Stripe gateway inside it, and prefer a `calculateDiscount` that returns a `Discount` over an `applyDiscount` that mutates the cart [codebase-design § Designing for testability].

Seam placement rules across his skills:

- Tests live at seams, never against internals, and the seams are written down and confirmed with the user before any test is written [tdd § Seams: where tests go].
- Prefer existing seams and use the highest seam possible; fewer seams is better and the ideal is one [to-spec § Process].

- A seam too shallow to reproduce the real call pattern gives false confidence [diagnosing-bugs § Phase 5: Fix + regression test].
- His job as the human is to force agents to use the small interface rather than reach into implementation details [fixing-the-pr-bottleneck-matt-pocock-aihero @ 00:09:10].

His cautionary example is a test that stubs the DOM audio context with fake methods, so the real API's error modes can never fail it [fixing-the-pr-bottleneck-matt-pocock-aihero @ 00:07:30].

6. Interface design

When designing an interface, the skill asks three questions: can the number of methods be reduced, can the parameters be simplified, can more complexity be hidden inside [codebase-design § Deep vs shallow]? The interface includes invariants, ordering constraints, error modes, configuration and performance [codebase-design § Glossary], so a short signature with a hidden ordering rule ("call `init()` first") is a larger interface than it looks.

Inference: for each candidate interface, write down every fact a caller must know, not only the types. That list is the true size of the interface, and the thing to shrink.

7. Worked TypeScript examples

All code in this section is *Inference:* illustration of the cited rules, not code Matt published.

7.1 Collapsing a pass-through cluster

Before: three shallow modules a caller must sequence correctly.

```ts
// cart/getLineTotals.ts
export const getLineTotals = (items: LineItem[]) =>
  items.map((i) => i.unitPrice * i.quantity);

// cart/sumTotals.ts
export const sumTotals = (totals: number[]) =>
  totals.reduce((a, b) => a + b, 0);

// cart/applyCoupon.ts: must run after sumTotals, before tax
export const applyCoupon = (subtotal: number, coupon?: Coupon) =>
  coupon ? subtotal * (1 - coupon.percentOff / 100) : subtotal;

// every caller repeats the sequence
const subtotal = applyCoupon(sumTotals(getLineTotals(cart.items)), cart.coupon);
const total = subtotal + tax(subtotal, region);
```

The deletion test: delete `sumTotals` and its one line reappears in each caller unchanged, so it was a pass-through. The ordering rule lives in a comment, which makes it part of the interface [codebase-design § Glossary] while nothing checks it.

After: one module, one function, one result.

```ts
// pricing/index.ts: the whole interface
export type Quote = {
  subtotal: Money;
  discount: Money;
  tax: Money;
  total: Money;
};

export function quote(cart: Cart, region: Region): Quote {
  // line totals, coupon ordering, rounding and tax live here
}
```

Callers learn one function. The ordering rule now sits behind the interface, so it is local: fixed once, fixed everywhere [codebase-design § Glossary]. The old unit tests on `sumTotals` and its neighbours are replaced by tests of `quote`, which is the "replace, don't layer" rule in the locally installed copy of the skill's DEEPENING.md (not stored in the brain).

7.2 A seam that earns its keep

```ts
// payments/port.ts
export interface PaymentGateway {
  charge(amount: Money, token: string): Promise<ChargeResult>;
}

// checkout/index.ts: deep module, dependency accepted, result returned
export async function checkout(
  cart: Cart,
  deps: { gateway: PaymentGateway; clock: () => Date },
): Promise<CheckoutResult> {
  /* validation, idempotency, retries, receipt building */
}
```

Two adapters exist, a Stripe adapter in production and an in-memory fake in tests, so by Matt's rule the seam is real [codebase-design § Principles]. It follows the skill's `processOrder` example of accepting the gateway [codebase-design § Designing for testability]. If Stripe were the only adapter and no test used a fake, `PaymentGateway` would be a hypothetical seam: indirection with nothing varying across it.

7.3 Narrowing the state a caller must handle

```ts
// Shallow: every caller must know which field combinations are legal
type Upload = { status: string; progress?: number; url?: string; error?: Error };

// Deeper: illegal combinations cannot be expressed
type Upload =
  | { status: "uploading"; progress: number }
  | { status: "done"; url: string }
  | { status: "failed"; error: Error };
```

This applies Matt's discriminated-union tip [you-have-to-know-this-typescript-pattern @ 00:00:49] to interface size: the union moves an invariant out of each caller's head and into the type.

7.4 A reducer as a deep module for UI state

```ts
export type Action =
  | { type: "added"; item: LineItem }
  | { type: "removed"; id: string }
  | { type: "couponApplied"; code: string };

export function cartReducer(state: CartState, action: Action): CartState {
  /* every cart rule lives here */
}

// tested with no browser
expect(cartReducer(empty, { type: "added", item })).toEqual(expected);
```

This follows his reducer short: the reducer returns the next state, is testable outside the browser, and keeps the React code simple [the-secret-to-building-robust-frontend-apps-with-agents § Detailed source notes].

8. Anti-patterns

Anti-pattern	Why Matt's sources reject it	Citation
Many tiny shallow modules	Looks modular; hard for the AI to explore	[software-fundamentals-matter-more-than-ever @ 00:13:26]
Pure functions extracted only for testability	Real bugs hide in how they are called; a locality failure	[improve-codebase-architecture § Process › 1. Explore]
A seam with one adapter	Hypothetical; nothing varies across it	[codebase-design § Principles]
Internal parts exposed through the interface	Depth belongs to the interface	[codebase-design § Principles]
Testing past the interface	The module is probably the wrong shape	[codebase-design § Principles]
Mocking internal collaborators, testing private methods, checking via the database	Implementation-coupled tests	[tdd § Anti-patterns]
Stubbing an external API so its error modes cannot fail the test	Makes the check cheatable, even in good faith	[fixing-the-pr-bottleneck-matt-pocock-aihero @ 00:07:30]
Depth as implementation lines over interface lines	Rewards padding	[codebase-design § Rejected framings]
Gray-boxing critical code such as finance	He would still review the implementation	[software-fundamentals-matter-more-than-ever @ 00:15:53]
Letting the agent pick and run refactors unsupervised	The human is the general who decides	[how-to-de-slop-a-codebase-ruined-by-ai § Detailed source notes]

9. Checklist

Synthesis of the cited rules above, phrased as checks:

- [ ] Scope to recently changed code, or to a direction the user named.
- [ ] Read `GLOSSARY.md` and the ADRs in the area before judging.
- [ ] Run the deletion test on each suspect module: does complexity vanish, or reappear across callers?
- [ ] List each module's whole interface: types, invariants, ordering, errors, configuration, performance.
- [ ] Look for concepts that need many small modules to understand.
- [ ] Look for rules duplicated across tiers or callers.
- [ ] Rate each candidate Strong, Worth exploring or Speculative, and let the human pick one.
- [ ] Only after the pick, design the interface (consider designing it twice).
- [ ] Put the seam as high as possible, aim for one, and confirm it with the user.
- [ ] Add a port only when two adapters exist (usually production and test).
- [ ] Replace old shallow-module tests with tests at the new interface.
- [ ] Record new terms in the glossary and load-bearing rejections as ADRs.
- [ ] For critical modules, still review the implementation.

10. Dates

- **2026-02-26**: earliest corpus video on deep modules, gray boxes and matching the file system to the mental map [your-codebase-is-not-ready-for-ai § Detailed source notes].
- **2026-04-23**: the AI Engineer talk lays out complexity, entropy, ubiquitous language, deep modules and gray boxes [software-fundamentals-matter-more-than-ever § Detailed source notes].
- **2026-04-29**: the architecture skill has gained a glossary of terms, to give developer and model a common vocabulary [how-to-de-slop-a-codebase-ruined-by-ai @ 00:01:14].
- **2026-08-19**: last-modified date of the separate `codebase-design` skill [codebase-design § Original source frontmatter].
- **2026-09-26**: the review talk refers to the skill for deepening opportunities and moves coding standards into a separate review sub-agent [fixing-the-pr-bottleneck-matt-pocock-aihero § Detailed source notes].

11. Limits of this report

- Matt's published work in this corpus doesn't cover a numeric threshold for "shallow"; the skill deliberately avoids a line-count ratio [codebase-design § Rejected framings].
- The brain does not store the skill's linked DEEPENING.md and DESIGN-IT-TWICE.md [codebase-design § Going deeper]. The one "replace, don't layer" mention comes from the locally installed copy, not the brain.
- Views on narrow interfaces and dependency checkers in the Robert C. Martin stream are Martin's, not Matt's [live-uncle-bob-software-fundamentals-ai @ 00:27:37].
- Nothing here was checked against current TypeScript, Node or React behaviour.